Trabajo Práctico Especial - Redes de Información

ITBA - 2C 2026

Tema: Gestión de Infraestructura por Código con Terraform

Integrantes:

Matias Rossi
Juan Pablo Birsa
Federico Viera

Índice

1. **Contexto y Problemática** 2
2. **Diseño de la Solución** 2
3. **SCOPE del POC y casos de uso** 3
4. **Diagrama de Arquitectura** 4
5. **Alternativas consideradas** 5
6. **Fuentes** 6

1. Contexto y Problemática

Toda aplicación necesita, además de su código, una infraestructura sobre la cual ejecutarse: redes con su direccionamiento, servidores, puntos de entrada, bases de datos y reglas de filtrado. Cuando esa infraestructura se arma a mano, con clics en una consola o con una secuencia de comandos, aparecen tres problemas conocidos: los entornos se vuelven irrepetibles, porque cada servidor termina siendo un ejemplar único configurado de memoria (lo que Fowler denominó *snowflake server*); no queda registro de qué se cambió, cuándo ni por qué; y la recuperación ante una falla depende de que alguien recuerde los pasos exactos. Estas limitaciones motivaron la aparición de las herramientas de Infraestructura como Código (IaC), entre ellas Terraform.

The Store es una plataforma de comercio electrónico compuesta por cinco microservicios (UI, Catalog, Cart, Orders y Checkout) escritos en distintos lenguajes (Java, Go y Node.js), cada uno con su propio motor de persistencia. La cátedra la provee junto con un script de bash (`local.sh`) que crea un clúster de Kubernetes local con kind, construye las imágenes de los cinco servicios, las carga en el clúster y aplica un manifiesto YAML con todos los objetos.

Ese script resuelve el arranque, pero arrastra las limitaciones propias de un enfoque imperativo: describe una secuencia de pasos y no el resultado deseado. En concreto, no mantiene registro de lo que existe, por lo que sólo sabe crear todo o borrarlo todo; no detecta ni corrige las modificaciones hechas por fuera (si alguien altera el clúster con `kubectl`, nadie se entera); no permite revisar por adelantado qué se va a modificar antes de aplicar un cambio; y no contempla parametrizar el entorno (cantidad de réplicas, versiones, reglas de red). Llevado a un entorno productivo el problema se agrava, porque habría que crear además redes, subredes, clústeres, balanceadores y reglas de seguridad de forma manual.

Se propone entonces adoptar Terraform para declarar como código toda la infraestructura que The Store necesita, de manera versionable, auditable y reproducible.

2. Diseño de la Solución

Se utilizará Terraform (lenguaje HCL) para aprovisionar la totalidad del entorno, con los providers `docker` (red e imágenes), `kind` (clúster de Kubernetes), `kubernetes` (objetos que corren dentro del clúster) y `helm` (instalación del controlador de Ingress), más `random` para generar credenciales.

El POC se despliega sobre un clúster local creado con kind (*Kubernetes IN Docker*), la misma plataforma que utiliza el script provisto por la cátedra. La decisión evita costos de nube y dependencias externas durante la demostración, y mantiene a la vista toda la capa de red (direccionamiento, NAT, ruteo entre nodos, resolución de nombres y reglas de filtrado), que es precisamente lo que interesa analizar en esta materia y que un proveedor administrado oculta detrás de su propia abstracción. El código se organiza de modo tal que migrar a una nube implique reemplazar los módulos `networking` y `cluster` y publicar las imágenes en un registry, sin tocar los módulos de bases de datos y de aplicación.

La infraestructura gestionada por Terraform se organiza en los siguientes módulos:

- **Networking:** red *bridge* de Docker con bloque 172.30.0.0/24 y su gateway, donde se conectan los nodos del clúster. Cumple el rol que en una nube cumpliría una VPC con sus subredes.
- **Cluster:** clúster de Kubernetes v1.35 con un nodo de control y dos nodos worker, con los bloques de direcciones de pods (10.244.0.0/16) y de servicios (10.96.0.0/16) declarados explícitamente, la publicación de los puertos 80 y 443 hacia la máquina anfitriona y la instalación del Ingress Controller (nginx), que actúa como balanceador de capa 7 y único punto de entrada. Equivale a un clúster administrado más su balanceador. La cantidad de nodos se fija al crear el clúster: kind no permite agregar nodos a un clúster existente, por lo que cambiarla implica recrearlo.
- **Databases:** las bases de datos de los microservicios que persisten información, MariaDB para Catalog, PostgreSQL para Orders y Redis para Checkout, desplegadas como StatefulSets con disco persistente. Las contraseñas las genera Terraform con el provider `random`, de modo que no hay credenciales escritas en el código. Equivalen a los servicios administrados de una nube (RDS y ElastiCache).

- **Application:** los cinco microservicios, cada uno con su Deployment (que mantiene N copias del proceso) y su Service (nombre DNS interno e IP virtual que balancea entre las copias); el Ingress que publica la tienda; y las NetworkPolicies que segmentan el tráfico siguiendo el principio de mínimo privilegio, cumpliendo el rol de los *security groups*.

La estructura de archivos será la siguiente:

```
terraform/
  main.tf                          # Orquestación de los módulos y construcción de imágenes
  variables.tf                  # Variables de entrada: réplicas, versión, redes, cuarentena
  outputs.tf                      # Resultados: URL de la tienda, kubeconfig, bloques de red
  providers.tf                                      # docker, kind, kubernetes y helm
  terraform.tfvars                                    # Valores concretos del entorno
  modules/
    networking/                                                 # Red y direccionamiento
    cluster/                                  # Clúster de Kubernetes e Ingress Controller
    databases/                                               # MariaDB, PostgreSQL y Redis
    application/                               # Microservicios, Ingress y NetworkPolicies
```

El estado de Terraform se mantiene en un archivo local, que ya provee bloqueo para evitar ejecuciones concurrentes. Para el trabajo colaborativo del equipo se deja documentada la migración a un backend remoto (bucket S3 con versionado y *locking*), que implica únicamente agregar un bloque `backend` sin modificar el resto del código.

3. SCOPE del POC y casos de uso

Scope del proyecto:

- Provisionar la red del clúster con el bloque 172.30.0.0/24 y su gateway.
- Desplegar un clúster de Kubernetes v1.35 con un nodo de control y dos nodos worker, con los bloques de pods y servicios declarados en el código.
- Instalar el Ingress Controller y publicar la tienda en `http://the-store.localhost`.
- Construir las imágenes de los cinco microservicios a partir del código fuente provisto y cargarlas en los nodos del clúster.
- Desplegar los cinco microservicios y las tres bases de datos con disco persistente.
- Aplicar reglas de segmentación de red entre las capas de frontend, backend y datos.
- Demostrar el plan de ejecución (`terraform plan`) previo a cada cambio, la creación completa (`terraform apply`) y la destrucción completa (`terraform destroy`) del entorno.
- Demostrar modificaciones de infraestructura mediante los casos de uso descritos a continuación.

Fuera de alcance: despliegue en nube pública, integración continua, observabilidad, gestión centralizada de logs, service mesh, certificados TLS y autoescalado, por corresponder a los temas de otros grupos o no ser necesarios para demostrar el objetivo del trabajo.

Casos de uso:

1. **Creación y destrucción completa del entorno de forma reproducible.** Se demuestra que toda la infraestructura puede crearse desde cero con `terraform apply` y eliminarse por completo con `terraform destroy`, sin dejar contenedores, redes ni volúmenes. Como complemento se demuestra la detección de *drift*: se modifica un objeto a mano con `kubectl` (por ejemplo, se cambia la cantidad de réplicas de un servicio), `terraform plan` muestra la diferencia entre el estado declarado y el real, y `terraform apply` la corrige. Es la capacidad que distingue a Terraform del script imperativo original.

2. **Escalamiento ante un pico de demanda.** La tienda necesita absorber un volumen de tráfico mayor al habitual. Se modifica el archivo `terraform.tfvars` para aumentar las réplicas de los microservicios más solicitados (por ejemplo, la UI de 2 a 4). Se ejecuta `terraform plan` para verificar el alcance del cambio y luego `terraform apply`; Kubernetes reparte las nuevas copias entre los nodos worker y el Service balancea el tráfico entre ellas. Se demuestra que un cambio de configuración en un único archivo escala la aplicación de forma controlada, sin intervenir el clúster a mano.

3. **Actualización de versión y rollback.** Se descubre un problema en la versión desplegada de la aplicación. Se modifica la variable que define la versión de las imágenes, se aplica el cambio y Kubernetes ejecuta un *rolling update* de los pods afectados sin interrumpir el servicio. Si la nueva versión introduce fallas, se revierte la variable al valor anterior y se vuelve a aplicar, demostrando la capacidad de volver atrás de forma controlada.

4. **Respuesta a un incidente de seguridad.** Se detecta que uno de los microservicios (por ejemplo, Checkout) fue comprometido. Como medida de contención se lo pone en cuarentena declarándolo en `terraform.tfvars`. `terraform plan` muestra que el único cambio es la capa a la que pertenece ese servicio, y al aplicarlo las reglas de segmentación dejan de permitirle cualquier comunicación: no recibe pedidos de la UI ni puede conectarse a otros servicios, a su base de datos ni a Internet. El resto de la tienda sigue operando (catálogo y carrito), y sólo la función del servicio aislado queda deshabilitada hasta resolver el incidente. Levantar la cuarentena es vaciar la lista y volver a aplicar.

4. Diagrama de Arquitectura

El entorno completo corre sobre una máquina con Ubuntu 24.04 y Docker. Terraform crea una red *bridge* de Docker con el bloque **172.30.0.0/24** y gateway 172.30.0.1, que funciona como un switch de capa 2 al que se conectan los nodos del clúster; el tráfico saliente hacia Internet se enmascara con la dirección de la máquina anfitriona (SNAT).

Cada nodo del clúster es un contenedor Docker basado en Debian 12 que ejecuta *containerd* y *kubelet*. Los tres nodos reciben direcciones consecutivas de esa red, asignadas por Docker al crear los contenedores (por ejemplo **172.30.0.2**, **.3** y **.4**). Dentro del clúster se usan dos bloques adicionales, disjuntos entre sí y respecto de la red de nodos y de la red local de la máquina: **10.244.0.0/16** para las direcciones de los pods (cada nodo recibe un /24 de ese bloque: 10.244.0.0/24 el nodo de control, 10.244.1.0/24 y 10.244.2.0/24 los workers) y **10.96.0.0/16** para las direcciones virtuales de los Services, que Kubernetes implementa con reglas de DNAT y que balancean el tráfico entre las copias de cada microservicio. La resolución de nombres interna la provee CoreDNS en la dirección 10.96.0.10. Las imágenes de los microservicios están basadas en Amazon Linux 2023; las de las bases de datos, en Ubuntu (MariaDB) y Alpine (PostgreSQL y Redis).

El nodo de control cumple además el rol de nodo de borde: es el único que publica puertos hacia la máquina anfitriona (80 y 443) y sobre él corre el Ingress Controller. Un pedido del navegador a `http://the-store.localhost` llega al puerto 80 del host, se reenvía por DNAT al nodo de borde, el Ingress Controller lo enruta según la cabecera `Host` hacia el Service de la UI, y éste lo entrega por DNAT a uno de los pods de UI en el puerto 8080. A partir de ahí, la UI consulta por HTTP a los cuatro microservicios de backend usando sus nombres DNS internos, y éstos consultan a sus bases de datos con los protocolos propios de cada motor: MySQL sobre el puerto 3306, PostgreSQL sobre el 5432 y RESP sobre el 6379.

La segmentación se aplica con NetworkPolicies siguiendo el principio de mínimo privilegio: por defecto ningún pod acepta conexiones entrantes; la UI sólo acepta tráfico proveniente del Ingress Controller; los microservicios de backend sólo aceptan tráfico de la UI y de otros backends; cada base de datos sólo acepta tráfico del microservicio que la usa; y un servicio en cuarentena no puede recibir ni iniciar conexiones. Todos los bloques utilizados son direcciones privadas y no se solapan entre sí.

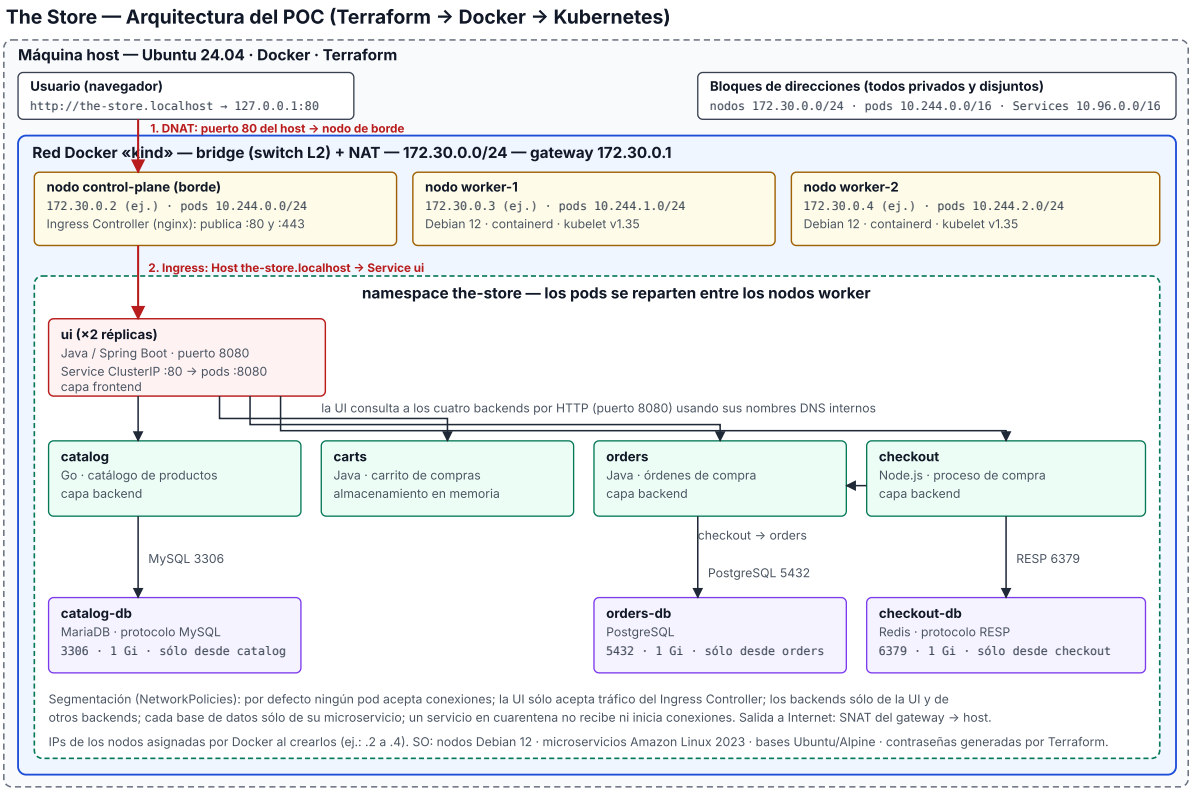


Figura 1: Arquitectura del POC: bloques de direcciones, nodos, microservicios, bases de datos y protocolos. En rojo el camino del tráfico del usuario; en negro el tráfico interno permitido por las NetworkPolicies.

5. Alternativas consideradas

Como alternativa a **Terraform** se consideró **Pulumi**, que permite declarar infraestructura con lenguajes de propósito general (TypeScript, Python, Go). Se optó por Terraform porque HCL es un lenguaje declarativo diseñado específicamente para configuración de infraestructura, lo que hace que el código sea más legible y que el plan de cambios sea directo de revisar, y porque su ecosistema de providers y su documentación son más maduros. También se evaluó **OpenTofu**, la bifurcación de código abierto de Terraform: es compatible con el código escrito, pero se mantuvo Terraform por ser la herramienta nombrada en el tema asignado.

Respecto de la plataforma, se consideró desplegar el POC sobre **una nube pública** (AWS, con VPC, EKS, RDS y ElastiCache). Se descartó por tres motivos: implica un costo por hora mientras el entorno está encendido, introduce una dependencia externa (credenciales, cuotas y conectividad) durante una demostración que es remota, y oculta detrás de servicios administrados buena parte de la capa de red que esta materia busca analizar. La estructura modular elegida deja preparado ese camino: los módulos de aplicación y bases de datos son independientes del proveedor.

Dentro de las opciones de Kubernetes local se evaluaron **minikube** y **k3d**. minikube levanta por defecto un único nodo, lo que impediría mostrar el ruteo de tráfico entre nodos; k3d utiliza k3s, una distribución reducida que incorpora su propio balanceador y controlador de Ingress y por lo tanto esconde componentes que interesa mostrar. Se eligió kind porque ejecuta Kubernetes sin modificaciones, permite declarar la topología y los bloques de direcciones, y es la misma herramienta que emplea el script provisto por la cátedra.

Finalmente se consideró resolver el despliegue únicamente con **Helm** o con **Ansible**. Helm y Kustomize sólo alcanzan a los objetos de Kubernetes: no crean la red, el clúster ni las bases de datos. Ansible está orientado a la configuración de servidores y no mantiene un estado con el que comparar la realidad, que es justamente la capacidad que se quiere demostrar. De todos modos ambos enfoques son complementarios: en esta solución Helm se invoca desde Terraform para instalar el Ingress Controller.

6. Fuentes

[1] HashiCorp. *What is Terraform?* y *The Core Terraform Workflow*. https://developer.hashicorp.com/terraform/intro

[2] Fowler, M. (2012). *SnowflakeServer*. martinfowler.com. https://martinfowler.com/bliki/SnowflakeServer.html

[3] Morris, K. (2020). *Infrastructure as Code: Dynamic Systems for the Cloud Age* (2ª ed.). O'Reilly Media.

[4] Kubernetes. *Services, Load Balancing and Networking* y *Network Policies*. https://kubernetes.io/docs/concepts/services-networking/

[5] kind. *Quick Start* y *Configuration*. https://kind.sigs.k8s.io/docs/user/quick-start/

[6] AWS Containers. *Retail Store Sample Application*. https://github.com/aws-containers/retail-store-sample-app